



Student

Management System

Introduction To Computing ~ LAB (Final Project)

PROGRAM DESCRIPTION

The **Student Management System** is a comprehensive **C++** application designed to manage student records for an educational institution. The system allows for:





1. Student registration with detailed personal and academic information
2. Administrative functions to view, search, update, and remove student records



3. Support for different academic levels
(Undergraduate, Graduate, Postgraduate)
4. Multiple program options (AI, CS, SE,
DS)



5. Automated scholarship calculations

based on academic performance

6. Hostel and transport facility

management



01

Key Features



1. MULTI-LEVEL STUDENT MANAGEMENT

- Undergraduate students (with intermediate marks tracking)
- Graduate students (with GPA tracking)
- Postgraduate students (with GPA tracking)

2. COMPREHENSIVE STUDENT INFORMATION

- Personal details (name, father's name, CNIC, gender, age, address)
- Contact information
- Academic details (program, level, department, semester)
- Performance metrics (marks or GPA)
- Facility requirements (hostel, transport)

3. ADMINISTRATIVE FUNCTIONS

- 
- View all student records in tabular format
 - Search students by name
 - Update student information

4. INPUT VALIDATION:

- 
- Comprehensive validation for all input fields
 - Case-insensitive input handling
 - Proper data type verification

5. USER-FRIENDLY INTERFACE:

- 
- An abstract graphic on the left side of the slide. It features several horizontal bars of varying lengths and colors (red, green, blue, yellow, and grey) stacked vertically. A large, light blue bracket is positioned to the right of these bars, pointing towards the list of interface features.
- Menu-driven navigation
 - Clear prompts and feedback
 - `System("cls")` for clean screen transitions



02

Features Used

1. CORE C++ FEATURES:

- Arrays for data storage
- Strings for text handling
- Control structures (loops, conditionals)
- Function (Setw)

2. STANDARD LIBRARIES:

- `<iostream>` for input/output operations
- `<string>` for string manipulation
- `<iomanip>` for formatted output (For Setw)

3. SYSTEM INTEGRATION:

- `system("cls")` for screen clearing



03

Core Mechanics

1) INPUT MANAGEMENT SYSTEM

Buffer Clearing (`cin.ignore()`)

Function: Resets the input channel before new entries

Why Needed: Prevents accidental input skips when switching between different input methods

Real Impact: Ensures every field gets proper attention, like a traffic light controlling data flow

Full-Line Capture (`getline()`)

Advantage Over `cin`: Captures complete sentences including spaces

Critical For: Address fields, multi-word names, detailed responses

Technical Note: Works in harmony with `cin.ignore()` to maintain input integrity

IN C++

```
cin.ignore(); // Clears buffer before  
  
getline()  
  
getline(cin, name[totalStudents]);
```

2) DATA VALIDATION

Numeric Input Validation

Checks if input contains only digits (for age, marks, etc.).

```
bool valid = true;
for (int i = 0; ageInput[i]; i++) {
    if (ageInput[i] < '0' || ageInput[i] > '9')
    {
        valid = false;
        break;
    }
}
```

String Input Validation

Ensures names contain only alphabets and spaces.

```
bool valid = true;
for (int i = 0; name[totalStudents][i]; i++) {
    char c = name[totalStudents][i];
    if (!((c >= 'a' && c <= 'z') || (c >= 'A'
&& c <= 'Z') || c == ' ')) {
        valid = false;
        break;
    }
}
```

CNIC & Contact Validation

Ensures exact length (13 digits for CNIC, 11 for contact)

```
bool valid = true;
int digits = 0;
for (int i = 0; cnic[totalStudents][i]; i++) {
    if (cnic[totalStudents][i] < '0' ||
        cnic[totalStudents][i] > '9') {
        valid = false;
        break;
    }
    digits++;
}
if (valid && digits == 13) {
    // Valid CNIC
}
```

Gender & Yes/No Input Validation

Converts input to lowercase for case-insensitive checks.

```
char g = gender[totalStudents][0];  
  
if (g >= 'A' && g <= 'Z') g += 32; // Convert to  
lowercase  
  
if (g == 'm') gender[totalStudents] = "Male";  
else if (g == 'f') gender[totalStudents] = "Female";
```

3) ARRAY-BASED DATA STORAGE

Parallel Arrays:

Uses multiple arrays to store student data (e.g., name[MAX], age[MAX], studentID[MAX]).

Limitation: Fixed size (MAX = 100), no dynamic resizing.

Student ID Counter:

Auto-increments for each new student:

IN C++

```
int idCounter = 1;
```

```
studentID[totalStudents] = idCounter++;
```

4) MENU-DRIVEN SYSTEM

Nested Switch-Like Logic

Uses if-else chains to simulate menu selections.

```
if (input == "1" || input == "s" || input ==  
"student") {  
    // Student menu  
} else if (input == "2" || input == "a" || input ==  
"admin") {  
    // Admin menu  
}
```

System Clear (system("cls"))

Clears console for better UI:

```
system("cls"); // Windows-specific
```

5) ARRAY-BASED DATA STORAGE

Display All Students

Uses formatted output with `setw()` for alignment.

```
cout << left << setw(10) << "ID" << setw(20)  
<< "Name" << setw(10) << "Semester";
```

Search by Name

Case-insensitive search.

```
bool match = true;

for (int j = 0; searchName[j] && name[i][j]; j++) {
    char c1 = searchName[j];
    char c2 = name[i][j];

    if (c1 >= 'A' && c1 <= 'Z') c1 += 32;
    if (c2 >= 'A' && c2 <= 'Z') c2 += 32;

    if (c1 != c2) match = false;
}
```

Update & Remove Students

Uses array shifting for deletion.

```
for (int i = index; i < totalStudents - 1; i++) {  
    name[i] = name[i + 1];  
    // Shift all other fields  
}  
totalStudents--;
```

6) SCHOLARSHIP CALCULATION

Based on marks (Undergraduate) or CGPA

(Graduate/Postgraduate)

```
if (level[i] == "Undergraduate") {  
    float perc = (interMarks[i] / 1200.0f) * 100;  
    if (perc >= 85) scholarship = "100%";  
    else if (perc >= 80) scholarship = "75%";  
    // ...  
}
```

Test Our Program

Enjoy !



Thanks !

For Listening